

Making Light Work of Lightcurve Fitting

User Manual

An interactive pipeline for fitting nova-like /
transient lightcurves with MCMC

A four-stage workflow:

Select → **Initialise** → **Fit** → **Plot**

June 13, 2026

Contents

1	Introduction	2
2	Installation and Environment	2
2.1	Requirements	2
2.2	Setting up	2
2.3	Files in the package	3
3	Quick Start	3
4	Data Format	3
5	Stage 1 — Region Selection	4
5.1	Controls	4
5.2	Undo, Redo and Delete	4
5.3	Axis scales	5
6	Stage 2 — Parameter Initialisation	5
6.1	Adjusting parameters	5
6.2	Finishing a region	5
6.3	Re-initialising	5
7	Stage 3 — MCMC Fitting	6
7.1	The progress and log windows	6
7.2	Sampler settings	6
7.3	Re-fitting	6
8	Stage 4 — Plots	7
9	Model Reference	7
9.1	Gaussian	7
9.2	Rising Exponential	7
9.3	Decaying Exponential	7
9.4	Bazin (rise-peak-fall)	7
9.5	Crystal Ball	7
9.6	Gaussian + Exponential Wings (composite)	8
10	Command-Line Reference	8
11	Files and Outputs	8
11.1	The regions file (<code>regions.json</code>)	8
11.2	Results directory	8
11.3	Plots directory	9
12	Troubleshooting	9
13	Tips	9

1 Introduction

This pipeline helps you fit analytic models to one-dimensional lightcurves (time vs. flux) without writing any code. You mark the regions of interest by clicking on the data, set up starting guesses with sliders, run a Markov Chain Monte Carlo (MCMC) fit, and produce publication-style plots. It is built around four stages that you normally run in order:

1. **Select** — interactively mark the time regions to fit and choose a model for each.
2. **Initialise** — set initial parameter guesses (and slider ranges) per region, optionally with an automatic least-squares pre-fit.
3. **Fit** — run the MCMC sampler (PyAutoFit + emcee) for each region and save the posterior samples.
4. **Plot** — generate the overview, per-region fit, and corner plots.

Everything is driven from a single command, `main.py`, with a `--stage` flag selecting which step(s) to run. The interactive stages open graphical windows; the fitting stage shows a live progress bar and a log window.

Note. The pipeline does not care about the physical units of your data. As long as time and flux are internally consistent, it will fit them. You are responsible for knowing what the axis values mean.

2 Installation and Environment

2.1 Requirements

The pipeline needs Python and a handful of scientific packages, listed in `requirements.txt`:

```
numpy>=1.24
scipy>=1.10
matplotlib>=3.7
corner>=2.2
autofit>=2024.1.1
emcee>=3.1
```

Python version. Python 3.11+ is recommended (PyAutoFit has first-class support for 3.12/3.13). Python 3.10 works but may print deprecation warnings.

Tkinter. The interactive stages need `tkinter`, which is part of the Python standard library and *cannot* be installed with `pip`:

- macOS / Windows (python.org or conda installers): already included.
- Linux: may need a system package, e.g. `sudo apt install python3-tk` (Debian/Ubuntu) or `sudo dnf install python3-tkinter` (Fedora).

2.2 Setting up

Using `conda` (recommended):

```
conda create -n lightcurve python=3.12
conda activate lightcurve
pip install -r requirements.txt
```

Or with a plain virtual environment:

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

2.3 Files in the package

You only ever *run* `main.py`, but all of the following `.py` files must sit together in the same folder:

File	Role
<code>main.py</code>	Command-line entry point; orchestrates the stages.
<code>selector.py</code>	Stage 1: interactive region selection.
<code>initialiser.py</code>	Stage 2: parameter initialisation (sliders).
<code>fitter.py</code>	Stage 3: MCMC fitting (PyAutoFit / emcee).
<code>plots.py</code>	Stage 4: overview / fit / corner plots.
<code>fitting_models.py</code>	Model definitions and priors.
<code>persistence.py</code>	Reading/writing JSON + saving MCMC results.
<code>progress_window.py</code>	GUI progress + log windows for fitting.
<code>ui_helpers.py</code>	Shared Tk/matplotlib widgets and dialogs.

3 Quick Start

Assuming your data file is at `~/data/lightcurve.txt`, the full workflow is just four commands:

```
python main.py --stage select --data ~/data/lightcurve.txt --regions
regions.json

python main.py --stage init --data ~/data/lightcurve.txt --regions
regions.json

python main.py --stage fit --data ~/data/lightcurve.txt --regions
regions.json \
--results results --walkers 60 --steps 1500 --burn 300

python main.py --stage plot --data ~/data/lightcurve.txt --regions
regions.json \
--results results --plots plots
```

You can also run every stage back-to-back with `--stage all`, though running them one at a time is clearer when you are learning the tool.

4 Data Format

By default the pipeline reads a whitespace-delimited text file with two or three columns:

Column 1	Column 2	Column 3 (optional)
time	flux	uncertainty

Lines beginning with `#` are treated as comments. If a third column is present it is used as the per-point 1σ uncertainty and the fit becomes properly weighted; otherwise an unweighted fit is performed.

To load a different format, edit the `load_data()` function near the top of `main.py`. It must return three arrays: `t`, `flux`, and `uncertainty` (or `None`).

Note. Within each region, time is shifted so that the region *start* is $t = 0$. Every fitted time parameter (peak position, t_0 , etc.) is therefore measured relative to the start of its region — exactly what the sliders show you in Stage 2.

5 Stage 1 — Region Selection

```
python main.py --stage select --data <file> --regions regions.json
```

A window opens showing your lightcurve. For each feature you want to fit:

1. Choose a model from the **dropdown** (top of the window).
2. **Click once** on the plot to set the region START (a green dashed line appears), then **click again** to set the END.
3. Optionally type a **note** (e.g. “rise”, “peak”) in the text box — this label is shown later when deleting regions.
4. Click **Confirm** (or press **Enter**) to save the region.

The **Confirm** button is greyed out until both clicks have been made, so you cannot accidentally save an incomplete region.

5.1 Controls

Action	How
Set region start / end	Left-click twice on the plot
Confirm region	Confirm button or Enter
Cancel an in-progress region	Esc
Undo last action	Undo button or u
Redo	Redo button
Delete a region by ID	d (opens a dialog)
Finish and save	q or close the window

5.2 Undo, Redo and Delete

Undo/redo cover *both* adding and deleting regions, using a proper action history. If you delete a region by mistake, just press **Undo** (or **u**) and it comes back exactly as it was — model, note, range and all. The **Undo** and **Redo** buttons grey out automatically when there is nothing to undo/redo.

Pressing **d** opens a dialog listing every region with its ID, model and note, so you can tell which number to delete, for example:

```
Existing regions:

#1: [rising_exp]  -- rise    (49291 -> 49409)
#2: [gaussian]   -- peak    (48808 -> ...)
#3: [decaying_exp]-- fall    (...)

Enter the segment ID to delete (Cancel to abort):
```

Watch out. Typing into the note box never triggers the single-key shortcuts (**u**, **d**, **q**, ...). The shortcuts are only active when the text box does not have focus.

5.3 Axis scales

Pass `--xscale log` and/or `--yscale log` to display (and store) logarithmic axes. The choice is recorded in the regions file and reused by the later stages.

6 Stage 2 — Parameter Initialisation

```
python main.py --stage init --data <file> --regions regions.json
```

Each region is opened in turn. The window shows the data, the current model curve (which updates live), the model equation in proper mathematical notation on the left, and one slider per parameter labelled by its symbol.

6.1 Adjusting parameters

- Drag a **slider** to change a parameter; the model curve redraws immediately.
- Each slider has **Minimum**, **Maximum** and **Step** boxes to its right (a small table with column headers). Edit these to change the slider's range or granularity.
- Click **Curve Fit** to run a quick least-squares pre-fit (`scipy.optimize.curve_fit`) from the current slider values. The result is drawn as a dashed line, and **Accept CF** / **Reject CF** buttons appear.
- **Accept CF** adopts the least-squares values as your guesses; **Reject CF** keeps the slider values.

6.2 Finishing a region

Action	How
Save guesses, go to next region	Confirm & Next
Skip this region	Skip
Undo / Redo a discrete action	Undo / Redo buttons
Quit (with confirmation)	q

If you press **Confirm & Next** while a curve-fit result is still unreviewed, a dialog asks whether to accept it, reject it, or cancel. Pressing **q** asks for confirmation before discarding the current region's work.

Note. Undo/redo here covers discrete actions — running a curve fit, accepting or rejecting it, and editing a min/max/step range — not continuous slider drags (to undo a drag, just drag back).

6.3 Re-initialising

If every region already has guesses, a dialog asks whether to re-do them or keep the existing values. To re-initialise specific regions directly, pass their IDs:

```
python main.py --stage init --data <file> --regions regions.json --ids
2 5
```

7 Stage 3 — MCMC Fitting

```
python main.py --stage fit --data <file> --regions regions.json \
  --results results --walkers 60 --steps 1500 --burn 300
```

This runs an `emcee` ensemble sampler (via `PyAutoFit`) for each initialised region. Uniform priors are built automatically around your initial guesses.

7.1 The progress and log windows

By default two windows open:

- **MCMC Fitting Progress** — an overall “region i of n ” bar plus a per-region bar that fills from 0 to 100% as the sampler runs. When fitting finishes it shows a green “*Fitting complete — click Close to continue*” message and enables the **Close** button.
- **Fitting log** — a scrolling copy of the messages that would otherwise print to the terminal.

Watch out. When the fit finishes, the program waits for you to **close the progress window** before the terminal returns to the prompt. This is intentional — it lets you read the final result first. Closing the window (red dot / `Cmd-W`) returns control to the terminal.

You can close the progress window *during* a fit; a confirmation appears, and the fit continues in the background and still saves its results.

To run without any windows (plain terminal output — useful for remote machines or scripts), add `--no-gui`:

```
python main.py --stage fit --data <file> --regions regions.json --no-gui
```

7.2 Sampler settings

Flag	Default	Meaning
<code>--walkers</code>	60	Number of <code>emcee</code> walkers.
<code>--steps</code>	1500	Number of steps per walker.
<code>--burn</code>	300	Steps discarded as burn-in.
<code>--force</code>	off	Overwrite an existing run directory.

Watch out. `--steps` must be greater than 100. `PyAutoFit`’s autocorrelation check analyses the chain minus its last 100 steps, so a shorter chain crashes inside `emcee`. The pipeline now refuses values ≤ 100 with a clear message; use a few hundred or more (the default 1500 is a good starting point).

Watch out. `--walkers` is capped at $50\times$ the number of model parameters. `emcee` only needs $\text{walkers} \geq 2\times \text{parameters}$; values far larger (e.g. a typo like 10000) waste compute and are rejected with a clear message. The default of 60 is plenty for these models.

7.3 Re-fitting

If a region already has output, the fitter refuses to overwrite it (so you cannot silently lose a good result). To re-fit from scratch, add `--force`. To fit only specific regions, use `--ids`:

```
python main.py --stage fit --data <file> --regions regions.json \
  --results results --ids 3 --force
```

8 Stage 4 — Plots

```
python main.py --stage plot --data <file> --regions regions.json \
  --results results --plots plots
```

This writes figures to the `--plots` directory:

- an **overview** plot with every fitted model overlaid on the full lightcurve;
- a **per-region fit** plot for each segment;
- a **corner** plot of the posterior for each segment.

Plots are saved to disk rather than shown interactively, so this stage is safe to run headless.

9 Model Reference

A model is chosen per region in Stage 1. In every formula below, t is *region-relative* time (zero at the region start), A is the amplitude and c is a constant baseline (`y_offset`). All timescale and width parameters (σ , τ) must be positive.

9.1 Gaussian

Symmetric peak on a flat baseline.

$$f(t) = A \exp\left[-\frac{(t - \mu)^2}{2\sigma^2}\right] + c$$

Parameters: amplitude A , centre μ , sigma σ , y_offset c .

9.2 Rising Exponential

$$f(t) = A e^{t/\tau_r} + c$$

Parameters: amplitude A , tau_rise τ_r , y_offset c .

9.3 Decaying Exponential

$$f(t) = A e^{-t/\tau_d} + c$$

Parameters: amplitude A , tau_decay τ_d , y_offset c .

9.4 Bazin (rise-peak-fall)

A single smooth function covering a full rise, peak and decay.

$$f(t) = A \frac{e^{-(t-t_0)/\tau_f}}{1 + e^{-(t-t_0)/\tau_r}} + c$$

Parameters: amplitude A , t0 t_0 , tau_rise τ_r , tau_fall τ_f , y_offset c .

9.5 Crystal Ball

Gaussian core with a power-law tail on one side. With $z = (t - \mu)/\sigma$:

$$f(t) = \begin{cases} A e^{-z^2/2} + c, & z > -|\alpha|, \\ A \left(\frac{n}{|\alpha|}\right)^n \frac{e^{-\alpha^2/2}}{\left(\frac{n}{|\alpha|} - |\alpha| - z\right)^n} + c, & z \leq -|\alpha|. \end{cases}$$

Parameters: amplitude A , centre μ , sigma σ , alpha α , n n , y_offset c .

9.6 Gaussian + Exponential Wings (composite)

A single, continuous rise–peak–fall profile that joins an exponential rise, a Gaussian peak and an exponential decay with no kinks. With $z = (t - \mu)/\sigma$, $a_L = \sigma/\tau_r$ and $a_R = \sigma/\tau_f$:

$$f(t) = c + A \times \begin{cases} \exp(a_L z + \frac{1}{2}a_L^2), & z < -a_L \quad (\text{rise}), \\ \exp(-\frac{1}{2}z^2), & -a_L \leq z \leq a_R, \\ \exp(-a_R z + \frac{1}{2}a_R^2), & z > a_R \quad (\text{fall}). \end{cases}$$

The join points and wing amplitudes are derived from σ , τ_r and τ_f so that both the value and the first derivative are continuous. Parameters: **amplitude** A , **centre** μ , **sigma** σ , **tau_rise** τ_r , **tau_fall** τ_f , **y_offset** c .

Note. Use this model when you want the entire outburst fit as *one* seamless curve with shared parameters, instead of fitting the rise, peak and decay as three independent regions.

10 Command-Line Reference

Flag	Default	Description
--stage	all	Stage to run: select , init , fit , plot , all .
--data	–	Path to the lightcurve data file.
--regions	regions.json	Regions JSON file.
--results	results	Output directory for MCMC results.
--plots	plots	Output directory for figures.
--ids	all	Restrict init/fit to these segment IDs.
--walkers	60	emcee walkers.
--steps	1500	emcee steps (> 100).
--burn	300	Burn-in steps to discard.
--force	off	Delete stale fit directories before fitting.
--no-gui	off	Fit with terminal output, no windows.
--xscale	linear	X-axis scale: linear or log .
--yscale	linear	Y-axis scale: linear or log .

11 Files and Outputs

11.1 The regions file (regions.json)

Produced by Stage 1 and updated by Stage 2. It stores, for each region: a numeric **segment_id**, the **start/end** times, the chosen **model**, an optional **note**, the **initial_guesses**, the **priors**, and any saved **curvefit_result**. It also records the axis scales. You normally never edit it by hand.

11.2 Results directory

For each fitted segment N , Stage 3 writes:

- **seg000N_samples.npy** — the posterior samples (after burn-in);
- **seg000N_lnprob.npy** — the log-probability of each sample;
- **seg000N_summary.json** — median values and asymmetric uncertainties per parameter.

PyAutoFit also keeps its own internal output under a **pyautofit/** sub-directory.

11.3 Plots directory

Overview, per-region fit, and corner plots, written as image files by Stage 4.

12 Troubleshooting

“n_steps is too small” — you passed `--steps` ≤ 100 . Use more (the default is 1500).

“n_walkers is excessive” — you asked for far more walkers than the model needs. Use something near the default of 60.

“PyAutoFit output directory already exists” — a previous (possibly crashed) run left output for that segment. Add `--force` to overwrite it, or use `--ids` to fit other segments.

“Skipping un-initialised regions” — those regions have no guesses yet. Run Stage 2 (`--stage init`) for them first.

The terminal seems stuck after a fit — the progress window is still open. Close it and the prompt returns. The fit itself is already finished and saved (look for “Done.” and the saved file paths in the log).

Dialog appears on the wrong monitor — pop-up dialogs anchor to the mouse pointer, so move your cursor to the screen you want before triggering them.

Nothing happens when I type u/d/q — a text box probably has focus. Click on the plot (or elsewhere) first; single-key shortcuts are suppressed while typing.

13 Tips

- Run the stages one at a time while learning; switch to `--stage all` once comfortable.
- Use separate `--regions`, `--results` and `--plots` names to keep different experiments apart.
- For the composite model, set μ near the peak and give σ a sensible width before running **Curve Fit** — the exponential wings hang off the Gaussian core, so the pre-fit converges best when the core is roughly in place.
- For a quick smoke test use `--steps 300`; for a publication fit use 1500 or more and the default 60 walkers.
- Few data points relative to parameters will produce harmless “Too few points to create valid contours” warnings on the corner plot — the fit itself is unaffected.